Redflag AI

Product Requirements Document | AI Factory — Native.builder Hackathon | Aug 3–10, 2026

1. Problem Statement

Freelancers and small agencies manage most client communication over unstructured channels — WhatsApp, email threads, Upwork/Fiverr messages. Two recurring risks hide inside that noise:

- **Scope creep** — a client casually asks for "just one more small thing" that isn't in the original agreement.
- **Payment risk** — vague or delayed language around invoices, timelines, or "let's discuss the budget later" that signals a payment problem forming before it becomes one.

Freelancers rarely catch these signals in the moment because they're read in passing, on mobile, between other client threads. By the time the pattern is obvious, the scope has already expanded for free or the invoice is already overdue.

2. Target User

Primary: Solo freelancers and small agency owners (2–5 people) doing client-based service work — design, development, marketing, content — communicating primarily over WhatsApp, email, or Upwork/Fiverr messaging.

Persona: A freelance developer or designer running 3–6 concurrent client relationships, without a dedicated account manager or CRM, who currently tracks scope and payment status manually or from memory.

3. Product Vision

An agent-powered workspace that reads incoming client conversation exports, flags scope-creep and payment-risk signals as they appear, drafts a professional boundary-setting or follow-up reply, and keeps a lightweight running log per client — so freelancers catch risk early instead of after the fact.

4. Core User Workflow (end-to-end, demoable in 3 minutes)

1. **Input** — User pastes or uploads a client conversation snippet (WhatsApp export, email thread, or Upwork message log) into the app.
2. **Classification agent** — Reads the conversation and classifies each message as: *Normal*, *Scope Creep Signal*, or *Payment Risk Signal*, with a one-line reason for the flag.
3. **Drafting agent** — For any flagged message, generates a suggested reply: a polite scope-boundary response ("Happy to help — that's outside our current agreement, here's a quick add-on quote") or a payment follow-up nudge, matched to the user's tone.
4. **Logging agent** — Writes a short structured entry to a per-client log: date, flag type, summary, whether a reply was sent.

5 **Output** — User sees flagged messages, suggested replies, and a running per-client risk history in one dashboard view.

5. Feature Scope

Must-have (hackathon MVP)

- Paste-in text input for a conversation (no live chat integration needed for MVP)
- Classification agent: flags scope creep / payment risk / normal
- Drafting agent: one suggested reply per flagged message, editable before sending
- Per-client log view: chronological list of flags and outcomes
- Deployed, publicly accessible URL via native.builder

Nice-to-have (if time allows)

- Multiple client "profiles" so the log separates by client
- Bright Data live-web lookup: pull a client's public business context (company site, reviews) to calibrate risk tone — e.g., an established company vs. a brand-new one changes how firmly to word a boundary-setting reply
- Simple risk score per client (count of flags over time)

Out of scope for hackathon

- Direct WhatsApp/email API integration (paste-in is sufficient for demo)
- Payment processing or invoicing itself
- Multi-user/team accounts

6. Technology Approach

Layer	Approach
App shell, UI, workflow logic	native.builder (primary requirement)
Classification + drafting agents	native.builder's built-in AI agents
Optional: live client context lookup	Bright Data MCP — pulls public info about a client/company to calibrate reply tone
Data storage	native.builder's built-in data/backend layer for per-client logs

This targets the **Bright Data "Best Agentic Use" partner prize** if the live-web lookup feature is built, since it's a genuine agentic use of live data (contextual risk calibration) rather than a static scrape.

7. Success Metrics (for the demo, not production)

- Correctly classifies at least 3 distinct signal types in a sample conversation (normal, scope creep, payment risk)
- Produces a usable, editable draft reply for each flagged message
- Full workflow — paste conversation → see flags → see draft reply → see logged entry — completes in under 60 seconds during the live demo

8. Judging Criteria Alignment

Criterion	How this project addresses it
Application of Technology	Multi-step agent pipeline (classify → draft → log), not a single prompt-response
Business Value	Solves a specific, recurring, real cost (unpaid scope creep, late payments) for a clearly defined user gro
Originality	Narrow niche (freelancer client-risk detection) vs. generic support-bot ideas
Presentation	Single clean before/after demo: messy conversation in, flagged risk + draft reply out

9. Demo Script Outline (for the 3-minute submission video)

1. (15s) State the problem: freelancers miss scope creep/payment signals buried in chat.
2. (30s) Paste in a realistic client conversation snippet.
3. (45s) Show the classification agent flagging 1–2 messages with reasons.
4. (45s) Show the drafting agent's suggested reply, edit it live.
5. (30s) Show the per-client log updating with the new entry.
6. (15s) Close: this is one workflow now, entirely inside a tool the user already has open.

10. Open Questions Before Building

- Do we seed the demo with real (anonymized) freelance conversation snippets, or construct realistic synthetic ones?
- Should the tone of drafted replies be configurable (firm / friendly / neutral), or fixed for MVP simplicity?
- Is the Bright Data live-context feature worth the build time, or does it risk pulling focus from the core flagging workflow?